

Introduction

This unit discusses The examples we have seen so far in previous units deal with standard input and output. When data is stored using variables, data is lost when the program exits unless something is done to save it. This unit discusses methods of working with files, and a data structure to store data.

C views file simply as a sequential stream of bytes. Each file ends either with an end-of-file marker or at a specified byte number recorded in a system maintained, administrative data structure.

C supports two types of files called binary files and text files. The difference between these two files is in terms of storage. In text files, everything is stored in terms of text i.e. even if we store an integer 54; it will be stored as a 3-byte string - "54\0". In a text file certain character translations may occur. For example a newline(`\n`) character may be converted to a carriage return,



linefeed pair. This is what Turbo C does. Therefore, there may not be one to one relationship between characters that are read or written and those in the external device.

A binary file contains data that was written in the same format used to store internally in main memory. The fact that a numeric value is in a standard length makes binary files easier to handle. No special string to numeric conversions is necessary.

The disk I/O in C is accomplished through the use of library functions. The ANSI standard, which is followed by TURBO C, defines one complete set of I/O functions. But since originally C was written for UNIX operating system, UNIX standard defines a second system of routines that handles I/O operations.

Key Concepts

Files provide persistent storage for data beyond program execution

Two main types: **text files** and **binary files**

Standard Input/Output System

There are no in-built input or output statements in C language. This means that all input and output must be done by calling functions, at least some of which must be written in a language other than C.

There is, however, a standard library of functions that allow I/O to be done in a relatively uniform manner for all C implementations without the need to know how it is being done in any particular case.

This standard library is known as "stdio" library and can be considered to be an extension to the C language itself. To be able to use functions within this library it is necessary to insert the following preprocessor statement at the start of the program source code:

```
#include <stdio.h>
```



Common stdio Functions



getchar

Reading single characters from standard input



gets

Reading a whole line of characters from standard input



putchar

Writing single characters to standard output



printf

Writing more complex, formatted output to standard output

Functions getchar, putchar and printf are used extensively in the program and many other input and output.

File Operations

A sequential stream of bytes ending with an end-of-file marker is what is called a file. When a file is opened, stream is associated with the file. By default, three files and their streams are

automatically opened when program execution begins – the standard input, standard output, and standard error.

Streams provide communication channels between files and programs. For example, the standard input stream enables a program to read data from the keyboard, and the standard output stream enables to write data on the screen. Opening a file returns a pointer to a FILE structure (defined in `stdio.h`) that contains information, such as size, current file pointer position, type of file etc., to perform operations on the file.

This structure also contains an integer called a file descriptor which is an index into a table maintained by the operating system namely, the open file table. Each element of this table contains a block called file control block (FCB) used by the operating system to administer a particular file.



FILE Structure

The standard input, standard output, and standard error are manipulated using file pointers `stdin`, `stdout`, and `stderr`.

